

Inventory Management System

A design for eliminating unexplained inventory loss by making every unit traceable from the moment it enters the warehouse to the moment it leaves.

Version 0.1

2026-07-27

Status: Draft

15 sections

CONTENTS

- | | | | |
|----|----------------------------|----|-------------------------|
| 01 | Executive summary | 09 | Audit & investigation |
| 02 | Architecture overview | 10 | Dashboard KPIs |
| 03 | Core domain model | 11 | Reports |
| 04 | Products, variants & units | 12 | Loss-reduction features |
| 05 | The inventory ledger | 13 | Future integrations |
| 06 | Stock in workflows | 14 | Tech stack |
| 07 | Stock out workflows | 15 | Phased rollout |
| 08 | Roles & approvals | → | Open questions |

01 Executive summary

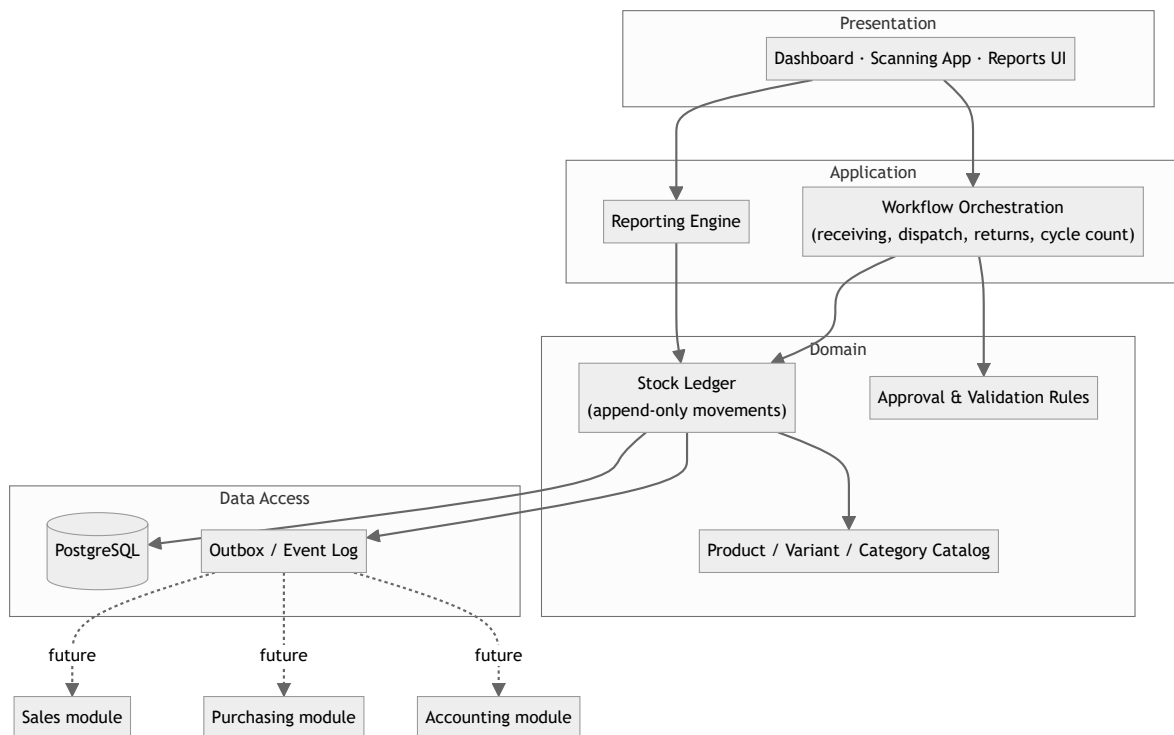
The client's core business problem is that inventory units go missing without anyone being able to say **when**, **who**, or **which transaction** caused it. Every design decision here is subordinate to that objective.

The single most important architectural choice is in §5 — The Inventory Ledger: stock levels are never stored as a number that gets directly edited. They are always the sum of an immutable history. Everything else — workflows, dashboard, reports, roles — is built on top of that ledger.

Secondary objectives — dynamic products/categories/variants, a dashboard, exportable reports, and room to plug in Sales/Purchasing/Accounting/Warehouse modules later — are addressed in the sections that follow, but none of them are allowed to compromise the ledger's integrity.

02 Architecture overview

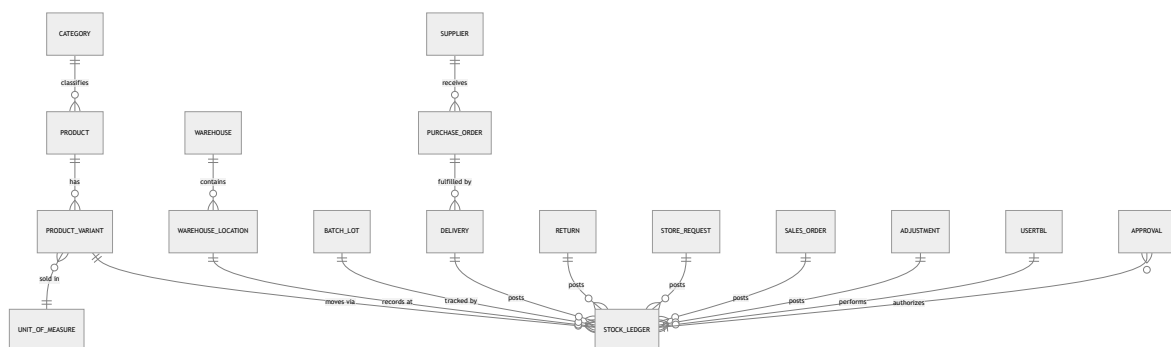
A **modular monolith**, not microservices — one system, internally layered, so a future split into standalone Sales/Purchasing/Accounting services doesn't require a rewrite.



Dependencies point inward only: presentation depends on application, application depends on domain, and nothing in the domain layer knows about HTTP, React, or PDF generation. That's what lets Sales/Purchasing/Accounting/Warehouse be added later as consumers of the same domain events instead of bolt-ons that reach into the database directly (§13).

03 Core domain model

Five table families sit at the center of the schema. Full field-level detail follows in later sections — this is the map.



Every arrow into `STOCK_LEDGER` runs one direction and append-only — a delivery, return, dispatch, or adjustment *creates* ledger rows; none of them ever edit a row that already exists.

Dynamic categories

`category` is self-referencing (`parent_category_id`) so users create unlimited categories and subcategories without a developer. Categories are **archived, never hard-deleted** — one used by historical transactions can't disappear out from under a report.

Products vs. variants

The client's own example — **Tiny**, sold as Per Sack / Per Ream / Per Piece — is the standard "one product, many sellable units" problem. The right structure separates three things that are easy to conflate: what the item *is* (product), what unit it's physically counted in (base unit), and what unit it's *sold* in (variant + conversion ratio).

TABLE	PURPOSE	KEY FIELDS
product	The conceptual item	<code>id</code> , <code>name</code> , <code>description</code> , <code>category_id</code> , <code>base_unit_id</code> , <code>supplier_id</code> , <code>status</code>
unit_of_measure	Canonical unit list	<code>id</code> , <code>code</code> , <code>name</code>
product_variant	A sellable form of a product	<code>id</code> , <code>product_id</code> , <code>sku</code> , <code>barcode</code> , <code>unit_id</code> , <code>conversion_to_base</code> , <code>cost</code> , <code>selling_price</code> , <code>status</code>
stock_balance	Cached, derived on-hand qty — always base unit	<code>product_id</code> , <code>warehouse_location_id</code> , <code>batch_id</code> , <code>qty_on_hand</code>

WHY BASE-UNIT ONLY

If Per Sack, Per Ream, and Per Piece each kept independent stock counters, a sale of one sack and a sale of 40 loose pieces would need manual reconciliation to agree on what's actually left — that gap is exactly how "missing inventory" complaints start. Instead, every variant declares a `conversion_to_base` (1 Sack = 50 kg, 1 Ream = 500 Pieces...), every movement converts to base-unit quantity before posting to the ledger, and breaking a larger unit into a smaller one is itself a logged `UNIT_CONVERSION` ledger transaction — never a silent inference.

Each variant keeps its **own** SKU, barcode, and selling price so retail and wholesale pricing/scanning work per sellable unit, while inventory truth stays unified underneath.

Recommended additional product fields

FIELD	WHY
reorder_point / reorder_qty	Drives Low Stock KPI and a future auto-PO trigger to Purchasing
min_stock / max_stock	Enables both Low Stock and Overstock exception reporting
is_lot_tracked	Forces batch/lot capture at receiving for perishables/regulated goods
is_serialized	Flags theft-prone/high-value SKUs for unit-level serial tracking (§12)
default_warehouse_location_id	Speeds up putaway suggestions
weight / dimensions	Needed later for carrier/shipping integration
tax_category	Needed later for Accounting/invoicing integration
image_url	Warehouse staff visually confirm the right item, reducing pick errors

05

The inventory ledger

This is the load-bearing decision in the whole system.

Rule: no table anywhere stores a stock quantity that application code increments or decrements directly. Every unit that enters or leaves the warehouse — delivery, return, dispatch, transfer, scrap, or manual adjustment — creates one row in `stock_ledger`. Current stock is always a derived sum, never a stored fact that can silently drift from reality.

FIELD	TYPE / VALUES
id	bigint PK
product_variant_id, warehouse_location_id, batch_id	FKs (batch nullable)
quantity_delta_base	numeric — + stock in, – stock out, always base unit
movement_type	PO_RECEIPT · CUSTOMER_RETURN · STORE_DISPATCH · WHOLESALE_DISPATCH · TRANSFER_IN/OUT · ADJUSTMENT_POSITIVE/NEGATIVE · SCRAP · CYCLE_COUNT_CORRECTION · UNIT_CONVERSION
reference_type, reference_id	polymorphic link to the source document
reason_code	required for adjustments and scrap
performed_by, approved_by	FK → users (approved_by nullable)
reversed_by_ledger_id	self-reference — corrections point here, never overwrite
created_at	immutable timestamp, never updated

What this buys, mapped to the client's stated needs

- **Prevents missing inventory** — stock can't become "wrong" through a UI bug or a forgotten update; there's no mutable number to forget updating.
- **Tracks every movement** — by construction. A movement that isn't a ledger row didn't happen; stock that changed without one is itself the anomaly a nightly reconciliation catches.
- **Fully auditable** — every row carries who, what, and when.
- **Corrections never destroy history** — fixing a mistake posts a reversing row rather than editing the original, so the mistake stays visible.

PERFORMANCE & SAFETY

A `stock_balance` table caches the summed total per product/location/batch, updated in the same DB transaction as the ledger insert, so dashboards don't re-aggregate millions of rows on every read. A nightly job re-derives it from the ledger and alerts on any mismatch.

Negative-stock guard: the app layer rejects any movement that would drive a balance below zero. A logged, approval-gated override exists for rare legitimate timing issues — every use of it surfaces on the dashboard, since it's a common way real shortages get hidden until a count finds them.

6.1 Supplier deliveries

FIELD	SOURCE	NOTES
Purchase Order	Given	Nullable — supports ad-hoc deliveries
PO line reference	ADDED	Needed for partial deliveries against a multi-line PO
Supplier	Given	
Delivery date (expected vs. actual)	Given + ADDED	Feeds the OTIF supplier KPI
Delivery Receipt No.	Given	System-generated fallback; sequential, gapless
Received By / Warehouse Location	Given	Location-level, not just warehouse-level
Product / Variant	Given	
Qty Ordered / Received / Accepted / Rejected	Given → SPLIT INTO FOUR	The gap between these four numbers is the discrepancy — one field would throw that information away
Batch/Lot + Expiry	Given + ADDED EXPIRY	Mandatory when lot-tracked; feeds FEFO picking
Cost / Landed cost	Given + ADDED	Freight & duties allocated across lines
QC status + Inspector	ADDED	Pass/Partial/Fail, separate from the receiving clerk
Discrepancy reason + photo	ADDED	Structured reason, not free text only

Status: Draft → Received → Inspected → **Posted to Stock** → Closed. Stock only posts at Posted — goods physically in the building but not yet inspected can't be picked against.

Validation: blocked without Product + Qty + Location + Received By; over-receipt beyond tolerance needs approval; lot-tracked categories require a batch number; zero-cost deliveries can't post.

6.2 Returned items

FIELD	SOURCE	NOTES
Original Sales Invoice	Given	Line-level link, not just order-level
Return Authorization No. (RMA)	ADDED	Issued before physical acceptance — blocks unauthorized walk-in returns
Customer / Return reason	Given	Structured reason list drives the Return Reason report
Condition of goods	Given → GRADED	Sellable / Damaged-Repairable / Damaged-Scrap / Expired
Quantity / Warehouse destination	Given	Destination driven by condition grade, not clerk judgment
Inspector / Date returned	Given	Must differ from RMA authorizer above a value threshold
Photo evidence	ADDED	Required when condition = Damaged

Status: Requested → RMA Authorized → Received → Inspected → Disposition → Closed.
 Stock restores to sellable balance only at Disposition — an unopened-looking box isn't stock until someone has looked inside.

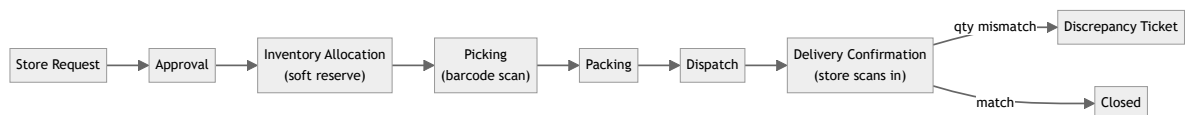
Validation: returns with no matching invoice are flagged for manual review, not blocked; cumulative returned quantity per line can't exceed original sold quantity.

07

Stock out workflows

Both channels share one principle: **every handoff between people is a barcode scan-to-confirm, not a typed number.** Typed re-entry of a quantity someone already scanned once is where transcription errors — and unaccountable shrinkage — creep in.

7.1 Retail store orders



Any mismatch at Picking, Dispatch, or Delivery Confirmation auto-opens a **Discrepancy Ticket** tied to the specific step and person who scanned it — the mechanism that answers "which transaction caused it."

7.2 Wholesale orders



Inventory is reserved at SO confirmation (so two customers can't be promised the same units) and lot-tracked stock allocates FEFO by default. Confirmed delivery is the trigger point for a future Accounting integration — the ledger doesn't need to know about invoicing, it just emits the event (\$13).

08 Roles & approvals

ROLE	CAN DO	CANNOT DO
Warehouse Staff	Scan receipts, pick, pack, dispatch	Post adjustments, edit cost/price
Warehouse Supervisor	Approve requests/variances within threshold	Approve their own adjustments
Inventory Controller	Post adjustments, run reconciliation, manage cases	Approve adjustments they posted themselves
Store Manager	Raise requests, confirm store deliveries	Approve own requests above auto-approve threshold
Sales Rep	Raise wholesale sales orders	Allocate inventory, approve credit exceptions
Approver / Ops Manager	Approve adjustments, no-invoice returns, overrides	Perform the action being approved
Admin	Manage users, categories, master data	Bypass the ledger — no such capability exists

CORE RULE

Maker-checker: the person who records a stock adjustment, scrap, or negative-stock override is never the person who approves it. This single rule closes the most common inventory-loss channel — unilateral power to both create and justify a "correction."

09 Audit & investigation design

CLIENT QUESTION	ANSWERED BY
When did the discrepancy happen?	stock_ledger.created_at
Who performed the transaction?	performed_by / approved_by
Which transaction caused it?	reference_type + reference_id
Expected vs. actual quantity	Captured at every handoff: pick, dispatch/delivery, and cycle count
Adjustment history	adjustment table, 1:1 linked to its ledger row
Approval history	approval table – requester, approver, timestamp, comment
Audit trail	The ledger itself, plus a generic <code>audit_log</code> for non-stock actions

RECOMMENDED ADDITION

Discrepancy Case: when a count or delivery variance exceeds tolerance, the system opens a formal case bundling expected qty, actual qty, variance value, every ledger entry in the window, an assigned investigator, and a required resolution — write-off, correction, or recovered — before any financial impact is booked. This turns "inventory went missing" from a one-line adjustment into a tracked, closeable investigation.

10

Dashboard KPIs

Requested, all included: Total Inventory Value · Total Products · Total Stock On Hand · Low Stock Items · Out of Stock Items · Returned Items · Stock In Today · Stock Out Today · Fast Moving Products · Slow Moving Products · Most Returned Products.

Recommended additions, prioritized by relevance to shrinkage

KPI	WHY IT MATTERS HERE
PRIORITY Shrinkage / Variance Rate (%)	Tracks the client's core pain point period over period — the single most important new number on this dashboard
Open Discrepancy Cases	Unresolved investigations before they age into write-offs
Negative-Stock Override count	Should sit near zero; any nonzero value is a red flag
Cycle Count Accuracy %	Leading indicator — degrading accuracy predicts future losses
Top Discrepancy Locations	Pinpoints where loss concentrates
Inventory Turnover / Days on Hand	Standard warehouse KPIs stakeholders will expect
Dead Stock Value	Capital tied up, distinct from "slow moving"
Pending Approvals	Surfaces bottlenecks in approval-gated workflows
Supplier OTIF Rate	Separates supplier short-shipment from warehouse-side loss

11

Reports

Exports: CSV, Excel, PDF. **Periods:** Daily, Weekly, Monthly, Custom range. **Exception highlighting** covers everything requested: missing inventory, discrepancies, negative stock, unusual movements, low stock, overstock, no-movement items, returns.

Recommended additional reports

REPORT	PURPOSE
Full Stock Ledger / Movement History	The master audit report — every movement, filterable by product, location, user, date, or type. What an auditor asks for first.
Adjustment Report	Every manual adjustment by user with approval status — surfaces a repeat pattern before it compounds
Cycle Count Variance Report	Expected vs. counted, by counter and location
Reconciliation Report	Ledger-derived balance vs. physical count — proves the ledger and reality agree
Return Reason Analysis	Root-causes return-driven loss instead of just logging it
Supplier Performance / OTIF	Separates supplier-side loss from warehouse-side loss

12 Loss-reduction features

Ranked by expected impact on the client's stated problem.

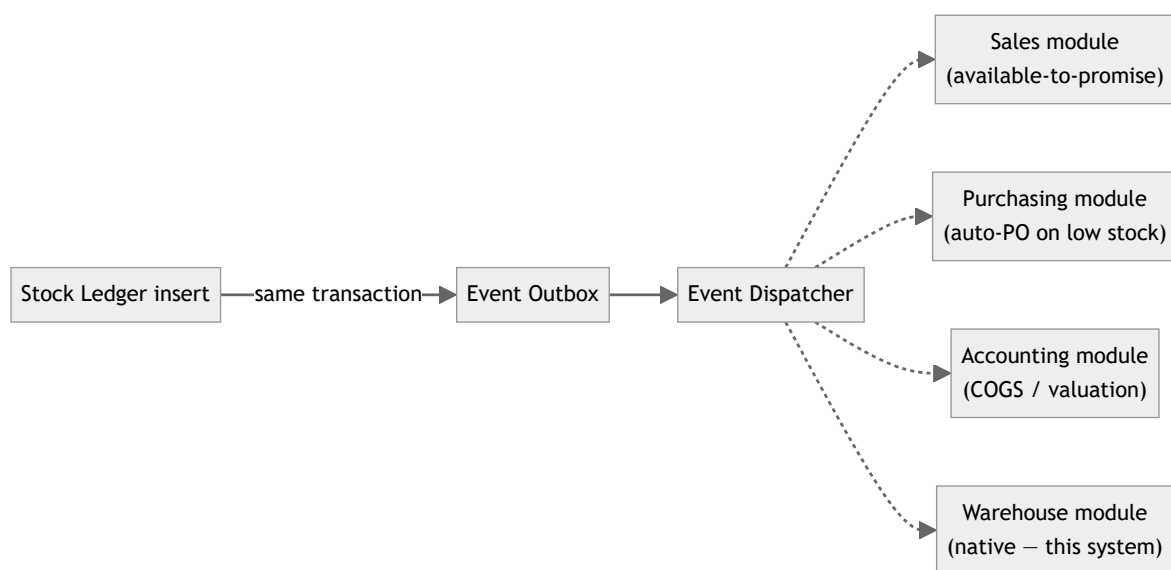
- 01 Mandatory barcode/QR scan-to-confirm at every touchpoint — receiving, putaway, pick, pack, dispatch, delivery, cycle count.
- 02 Append-only ledger with derived balances (\$5) — makes tampering or bugs detectable, not just harder.
- 03 Maker-checker on all adjustments and write-offs above a configurable threshold.
- 04 ABC-classified cycle counts — high-value/fast-moving items counted more often; variance auto-opens a Discrepancy Case.
- 05 Photo evidence at receiving, returns, and high-value dispatch.
- 06 Bin/location-level tracking — "missing from Bin A-14" is investigable; "missing from Warehouse 1" is not.
- 07 Hard block on negative stock, narrow logged override path only.
- 08 Serialized tracking for theft-prone/high-value SKUs, layered on batch/lot tracking for perishables.
- 09 Real-time low/zero-stock alerts — a genuine stockout shouldn't be mistaken for "missing" inventory.
- 10 Supplier scorecards — much "missing" stock at receiving is short-shipment, not warehouse theft.
- 11 Mobile-first scanning app for floor staff — entered at the point of action, not reconstructed later.
- 12 Sequential, gapless document numbering — reused or skipped numbers conceal discrepancies.
- 13 Nightly reconciliation job comparing derived ledger totals against the cached balance table.

14 Least-privilege, role-based access — ledger exports restricted and logged.

15 *Later-stage, optional:* RFID or shelf weight-sensors once barcode discipline is already mature.

13 Future integrations

Every ledger insert writes a row to an `event_outbox` table in the same transaction — the outbox pattern guarantees the event is never lost or double-fired independent of the ledger write.



All future modules integrate through this event stream and versioned internal APIs (`/api/v1/...`), never direct database access — each module owns its own data. Sales, Purchasing, and Accounting can be built later without touching the ledger's schema or logic.

14 Tech stack

LAYER	CHOICE	WHY
Frontend	Next.js + TS + Tailwind + shadcn/ui	Org default; fits dashboard + forms well
Backend	Next.js API routes → Fastify/Nest later	Start monolithic, split only for a real scaling reason
Database	PostgreSQL	Transactional integrity for the ledger is non-negotiable
ORM	Prisma	Versioned migrations, no hand-edited prod schema
Scanning	Web camera scan + handheld keyboard-wedge + mobile PWA	Avoids a native-app build for Phase 1
Reports	exceljs · headless PDF render · streaming CSV	Server-side, queued for large exports
Auth	Clerk / Auth.js + RBAC	Never hand-roll session/credential handling
Observability	Sentry + structured logs + correlation IDs	Every movement traceable end to end

15

Phased rollout

PHASE	SCOPE
1 — MVP	Ledger + balances; products/categories/variants/units; supplier receiving; retail + wholesale dispatch; roles & maker-checker; core dashboard; CSV/Excel export
2	Returns with disposition logic; ABC cycle counts; PDF reports; Discrepancy Case management; mobile scanning app
3	Event outbox + Sales/Purchasing/Accounting/Warehouse hooks; supplier scorecards; serialized tracking; fast/slow-mover analytics

Open questions for the client

1. Are Retail Store and Wholesale orders fulfilled from the **same** warehouse stock pool, or ring-fenced per channel?
2. Is FEFO mandatory for all lot-tracked goods, or only specific categories?
3. What adjustment value/quantity threshold should trigger mandatory approval (§8)?
4. Should the wholesale discrepancy-claim window (§7.2) be fixed, or vary by customer/contract?